

XRPL Lending Protocol · Developer Feedback

Hackathon DeVinci Blockchain × Ripple · 12 and 13 September 2026

CY-HACK

Track 1 · open-ended vault
Flavour Loaded: XLS-65, XLS-66, TokenEscrow

github.com/charlyppr/xrpl-lending

NETWORK

Custom Hackathon Devnet · `rippled` 3.4.0-rc1, hackathon branch · `LendingProtocolV1_1` and `fixCleanup3_4_0` enabled

LIBRARY

`xrpl@4.6.0`, for every finding · the 15 XLS-65/66 transaction types are typed · no raw JSON · one helper replaced (finding 1)

FIRST TRANSACTION

50 minutes, of which 45 on blocked ports (finding 8)

VERIFICATION

143 of 143 cited hashes validated on the ledger · docs checked against XLS-65, XLS-66 and `xrpl.org` · library claims against the `rippled` and `xrpl.js` source

#	Finding	Category	Severity	Library
1	<code>fixCleanup3_4_0</code> changes impairment timing and the <code>LoanSet</code> counterparty signature. <code>xrpl.org</code> does not list it, and the <code>xrpl.js</code> signing helper predates it.	client libraries	High	<code>xrpl@4.6.0</code>
2	A late <code>LoanPay</code> without <code>tfLoanLatePayment</code> returns <code>tecEXPIRED</code> . The <code>xrpl.org</code> <code>LoanPay</code> reference does not list this failure.	documentation/tutorials	High	<code>xrpl@4.6.0</code>
3	The first-loss capital covered 0.5 % of a defaulted loan. The broker then withdrew the remaining cover.	UX	High	<code>xrpl@4.6.0</code>
4	<code>xrpl.org</code> omits the sole-holder exception to share value. <code>LossUnrealized</code> is absent from the node while zero.	documentation/tutorials	Medium	<code>xrpl@4.6.0</code>
5	<code>loan_info</code> and <code>loan_broker_info</code> do not exist. <code>mpt_holders</code> needs <code>Clio</code> , which the devnet does not expose.	missing primitive	Medium	<code>xrpl@4.6.0</code>
6	Four result codes each cover situations that call for different actions.	UX	Medium	<code>xrpl@4.6.0</code>
7	The hackathon build reverts the V1.1 <code>LoanBrokerSet</code> restriction. The event brief does not mention it.	documentation/tutorials	Medium	<code>xrpl@4.6.0</code>
8	The devnet listens only on ports 51233 and 51234. A filtered port and a stalled node show the same signature.	other	Medium	<code>xrpl@4.6.0</code>
9	Closing a vault to deposits requires setting <code>AssetsMaximum</code> to <code>AssetsTotal</code> .	missing primitive	Low	<code>xrpl@4.6.0</code>
10	Vault shares carry <code>lsfMPTCanTrade</code> . <code>OfferCreate</code> with a share amount returns <code>temDISABLED</code> .	documentation/tutorials	Medium	<code>xrpl@4.6.0</code>

Severity. High: blocks a minimum bar step for any team, or misstates the risk a depositor carries. Medium: costs time or a wrong design decision, with a workaround. Low: documented elsewhere or cosmetic.

The six questions of the brief

- Vault, broker and loan.** Clear in the references: only the vault owner can submit `LoanBrokerSet`, so owner and broker are one account. The minimum bar's drawdown has no transaction: `LoanSet` moves the principal out of the vault.
- First-loss parameters.** No. `CoverRateLiquidation` applies to the minimum cover, not to the defaulted debt (finding 3).
- Multi-party `LoanSet`.** The broker signs, the borrower adds `CounterpartySignature` and submits. Both wallets ran in one script. The `xrpl@4.6.0` helper for the second signature is rejected on this devnet (finding 1).
- SDK or raw JSON.** All 15 types are typed. No raw JSON. One helper replaced (finding 1).
- Position value, utilisation, liquidity, yield.** Not from ledger objects: five calls and six values computed client-side (finding 5), fields absent while zero (finding 4).
- Explorer and docs against the ledger.** The docs differ on findings 1, 2, 4 and 10. The explorer shows `tfLoanLatePayment` in hex (finding 2) and no `Vault` field values in its *Detailed* tab (finding 4).

drawdown in `LoanSet` `D19CD59FB93CCF28547533C76CC1AFA71CF592FD0EC9D0A9E217A9F6EAA085DA`

1 · `fixCleanup3_4_0` is enabled and not listed on `xrpl.org`

The feature command reports `fixCleanup3_4_0` as enabled on the devnet. The `xrpl.org` *Known Amendments* page does not list it. The amendment changes two lending behaviours.

Impairment. XLS-66, section 3.10.4.2, condition 9: with `fixCleanup3_4_0`, `tfLoanImpair` fails while `currentTime <= NextPaymentDueDate`. The `xrpl.org` tutorial *Manage a Loan* says a broker can “manually impair a loan before a payment due date passes”.

Time relative to <code>NextPaymentDueDate</code>	<code>tfLoanImpair</code>
31 s before	<code>tecT00_S00N</code>
13 s before	<code>tecT00_S00N</code>
5 s after	<code>tesSUCCESS</code>
5 s after: <code>5486020BA8FF227C58409042F96BDC07246FDACC3F9B0EF24749555E319E972E</code>	

Counterparty signature. `rippled` PR #8162: with `fixCleanup3_4_0`, `CounterpartySignature` uses the CPT signing prefix. In `xrpl@4.6.0`, `signLoanSetByCounterparty` signs with `encodeForSigning`, the previous prefix, and has no option to change it. Its `LoanSet` fails local checks with `Counterparty: Invalid signature`. The same `LoanSet` signed with `encodeForSigningCounterparty` from `ripple-binary-codec` returned `tesSUCCESS`.

`23631C3610DA97926E8BF9FC5BC4E76E504C9DFB2683C928E825DE0138397B2B`

Proposed fix. List `fixCleanup3_4_0` on *Known Amendments* with its two lending changes. Update *Manage a Loan*: impairment is accepted once `NextPaymentDueDate` has passed. In `xrpl.js`, sign `CounterpartySignature` with `encodeForSigningCounterparty`, with the previous prefix as an option for networks without the amendment.

2 · A late `LoanPay` without `tfLoanLatePayment` returns `tecEXPIRED`

A loan 10 s past `NextPaymentDueDate`, within its grace period, rejected four `LoanPay` transactions with `tecEXPIRED`: the instalment, the instalment plus `LatePaymentFee`, the full balance, and a payment with `tfLoanFullPayment`. The same payment with `tfLoanLatePayment` (262144) returned `tesSUCCESS`.

- XLS-66, section 3.11.4.2, condition 11 defines this failure. The `xrpl.org` `LoanPay` reference lists eight error codes, and `tecEXPIRED` is not among them.
- The explorer shows `tfLoanImpair` and `tfLoanDefault` by name, and `tfLoanLatePayment` as `0x00040000`.
- No field of the `Loan` node changed during 85 s of lateness, and no field holds the accrued late interest. A payment of `ceil(PeriodicPayment) + LoanServiceFee + LatePaymentFee` returned `tecINSUFFICIENT_PAYMENT`. A

payment of 2 765 001 drops returned `tesSUCCESS`, and 765 003 drops were taken.

Proposed fix. Add `tecEXPIRED` to the `LoanPay` error table, with `tfLoanLatePayment` as the remedy. Name the flag in the explorer. Expose the amount due now, late interest included, in a `Loan` field or a read command.

Fail - TecEXPIRED

Status

This transaction failed with a status code of `tecEXPIRED`, and validated in ledger `67844` on September 12, 2026 at 2:33:42 PM UTC

Explorer. `LoanPay` of 1.030001 XRP without flags.

96C3870480E4CB0C9E3B925F2965347D0A815A486CDB4B9545928E32D6B0BA4

Success

Flags

0x00040000

Explorer. `LoanPay`, same loan, same amount, with `tfLoanLatePayment`.

EFAD383F98622357CE67CB8051809F95FC27BE611D43F26FD69427713FB608F
overpayment 9A7589384C5142703AFFACBB91B12E223EC734F7ACBE51272A87525FE4EE9CA8

3 · The first-loss capital covered 0.5 % of a defaulted loan

Both brokers used `CoverRateMinimum` 10 % and `CoverRateLiquidation` 5 %.

Vault	Loan	Cover	Taken	Loss	Share value
200 XRP	50	50	0.25	49.75	1.000 → 0.751
5 XRP	4	1	0.02	3.98	1.000 → 0.204

On both vaults the cover taken equals $\text{debt} \times 10\% \times 5\%$ to the drop. The [xrpl.org Lending Protocol](#) page gives this formula, with an example that covers 1 % of the debt. `CoverRateLiquidation` applies to the minimum cover ($\text{DebtTotal} \times \text{CoverRateMinimum}$), not to the defaulted amount. After a default, `DebtTotal` is zero and so is the cover floor.

Second vault, in ledger order:

UTC	Transaction	Result
14:23:50	<code>VaultWithdraw</code> , all shares	<code>tecINSUFFICIENT_FUNDS</code>
14:27:01	<code>LoanManage</code> <code>tfLoanDefault</code>	<code>tesSUCCESS</code>
14:27:11	<code>VaultWithdraw</code> , all shares, 1.02 XRP	<code>tesSUCCESS</code>
14:27:22	<code>LoanBrokerCoverWithdraw</code> , 0.98 XRP	<code>tesSUCCESS</code>

D91BA47E816FDA60F9BBAD7F100B383099D87021B08DE065E841B9E8E34AFE2D
3338BA629F8772CDEC07B6F714F2AF3A1ADC8E93642429A54B95CF372FDB2880
first vault default 91F458E3E6244E1C3005345C5C83CDFAF0E8A1AF6E5587E100C302CA56743BC7

Proposed fix. Next to the formula, state the share of a defaulted debt the cover absorbs: `CoverRateMinimum` × `CoverRateLiquidation`, 0.5 % at 10 % and 5 %. Hold `LoanBrokerCoverWithdraw` for a set delay after a default, or state that no delay exists.

Success

Flags

`tfLoanDefault`

- It modified the `AccountRoot` node of `rsR4QZgAQzJck3TdoT12duNL7bFCNtQAQJ`
 - Balance increased by $\times 0.02$ XRP from $\times 1.00$ XRP to $\times 1.02$ XRP
- It modified the `AccountRoot` node of `rbhF8rmZsXZfiacsNFX62psL9u3irY24H`
 - Balance decreased by $\times 0.02$ XRP from $\times 1.00$ XRP to $\times 0.98$ XRP

Explorer. `LoanManage` `tfLoanDefault` moves 0.02 XRP from the broker's pseudo-account (`rbhF8...`) to the vault's (`rsR4Q...`).

923F7D616B094C82197506CF2867907A93067041E3990D24BD07A19035F73EAC

Success

Amount

$\times 0.98$

Date/Time (UTC)

9/12/2026, 2:27:22 PM

Explorer. `LoanBrokerCoverWithdraw` of the remaining cover, 21 s after the default.

68DD97D949602FD470E0FA80B2526C3FDCFA3449095F7A7879F4D4430845D45E

4 · Share value and LossUnrealized

XLS-65 computes the assets returned on redemption as $\text{shares} \times (\text{AssetsTotal} - \text{LossUnrealized}) / \text{total shares}$, and waives the `LossUnrealized` deduction when the withdrawer is the sole outstanding shareholder. The [xrpl.org Single Asset Vault](#) page gives the formula and does not mention the exception.

- The `Vault` node has no `LossUnrealized` field until an impairment writes one. The impairment below moved it from absent to `"3000001"`, with `AssetsTotal` at `"5000002"`.
- `AssetsAvailable`, `AssetsMaximum`, `PaymentRemaining` and `TotalValueOutstanding` are also absent from their node when zero.
- The explorer's *Detailed* tab shows *"It modified a node with type Vault"* without field values. `LossUnrealized` appears in the *Raw* tab once the node is expanded.

Proposed fix. Add the sole-holder exception to the [xrpl.org](#) formula. Return zero-valued fields as `"0"` in `vault_info`, or document their absence. Show changed field values in the explorer's *Detailed* tab.

Success

- It modified a node with type `Loan`
- It modified a node with type `Vault`

`AssetsAvailable:` `"2000001"`

`AssetsMaximum:` `"100000000"`

`AssetsTotal:` `"5000002"`

`Flags:` `0`

`LEVersion:` `1`

`LossUnrealized:` `"3000001"`

Explorer. `LoanManage` `tfLoanImpair`: *Detailed* tab, then *Raw* tab with the `Vault` node expanded.

C8678C1C0A10BE889124F40C03017038654B278E2D464FAC54784F55B35318FC

5 · Read commands

`vault_info` is a rippled command. `loan_info` and `loan_broker_info` return `unknownCmd` and have no documentation page. [xrpl.org](#) documents `mpt_holders` as *Clio-only*. The devnet endpoints serve `rippled`, where `mpt_holders` returns `unknownCmd`. A depositor's view takes five calls:

Call	Returns
<code>vault_info</code>	vault totals, share issuance and share flags
<code>account_objects</code> <code>mptoken</code>	the depositor's shares
<code>account_objects</code> on the owner	the broker, filtered on <code>VaultID</code>
<code>account_objects</code> <code>loan</code>	the loans, on the broker's pseudo-account
<code>mpt_holders</code>	<code>unknownCmd</code>

Six values are computed client-side: gross and net share value, position value, withdrawable amount, utilisation and broker debt capacity.

Proposed fix. Add `loan_broker_info` and `loan_info`, or list a vault's brokers and loans in `vault_info`. Return share value, withdrawable amount and utilisation. Expose *Clio* on the devnet.

6 · Four result codes cover situations that call for different actions

Code	Situation	Hash
tecINSUFFICIENT_FUNDS	LoanSet above the cover limit	5DC3A6E0...4147618B
	LoanSet above the vault liquidity	B92A90F5...259F9CAA
	LoanBrokerCoverWithdraw below the floor	4FB4A2B0...262F1370
tecLIMIT_EXCEEDED	VaultDeposit above AssetsMaximum	132BAF7C...027B5B4F
	AssetsMaximum set below AssetsTotal	37053F15...A88C3DC6
	DebtMaximum set below DebtTotal	82C14ED4...748E4121
tecNO_PERMISSION	tfLoanImpair sent by the borrower	F0304194...5DE2C112
	tfLoanUnimpair on a loan that is not impaired	13975FD3...6AA24537
tecNO_AUTH	Payment of shares to an account without MPToken	789AAF7E...5FD512A4
	Payment of non-transferable shares	E9DE504D...B8BE5FD9

tecHAS_OBLIGATIONS names a single cause on VaultDelete, LoanBrokerDelete and LoanDelete.

Proposed fix. Where one transaction type maps two causes to one code (LoanSet, LoanManage, a Payment of shares), return distinct codes or a result message naming the check that failed.

❗ Fail - TecINSUFFICIENT_FUNDS

Status
This transaction failed with a status code of **tecINSUFFICIENT_FUNDS**, and validated in ledger **68327** on September 12, 2026 at 2:57:52 PM UTC

Explorer. LoanSet above the cover limit.

5DC3A6E00F31DA82C69AB77669C64691BD71CAC49C169736F53D7C724147618B

❗ Fail - TecINSUFFICIENT_FUNDS

Status
This transaction failed with a status code of **tecINSUFFICIENT_FUNDS**, and validated in ledger **64415** on September 12, 2026 at 11:42:11 AM UTC

Explorer. LoanSet above the vault liquidity. Same result code.

B92A90F59EBF79108B5414F61A12656F321EBC264E1BB186FCC14CAF259F9CAA

7 · The hackathon build reverts the V1.1 LoanBrokerSet restriction

The V1.1 documentation states “LoanBrokerSet is restricted on open-ended vaults”. The feature command reports LendingProtocolV1_1 as enabled. LoanBrokerSet returned tesSUCCESS on an open-ended vault and on a closed-ended vault. rippled PR #8076 added the restriction. The commit Revert "fix: Reject open-ended vaults at LoanBrokerSet (#8076)" removed it from the lending hackathon branch on 10 September 2026. The other V1.1 rules tested (new fields, date validation, phase locks) are enforced. The event brief does not mention the revert.

open-ended 781F54B5E77A768F4C02A54E3C55902AA9F1AC96AA04037AB97CE93FFB5DBDE4
closed-ended 1961BE21CC4011F50AA926E1494E9D51F94FC57B0E9C0C1B8F8ADF6FEDD09D40

Proposed fix. State in the event brief which V1.1 rules the hackathon build enforces, and that the #8076 restriction is reverted.

8 · Ports 51233 and 51234

The documented endpoints use ports 51233 (WebSocket) and 51234 (JSON-RPC). No endpoint is documented on port 443. On a Wi-Fi network, nc -z succeeded on both ports while curl on 51234 returned 0 bytes, and portquiz.net:51234 timed out. The same signature appeared five times during the event: three occurrences were

traced to the local network, two could not be attributed. The first cost 45 minutes. The xrpl.js Client uses WebSocket only. The faucet ignores the destination field and creates a new account.

Proposed fix. Serve JSON-RPC and WebSocket on port 443. Fund the destination account from the faucet instead of creating a new one.

9 · Closing a vault to deposits

The xrpl.org Vault entry and XLS-65 define AssetsMaximum: 0 as no cap. The VaultSet reference does not repeat it. VaultSet exposes Data, AssetsMaximum and DomainID, and no flag closes a vault to deposits. Setting AssetsMaximum equal to AssetsTotal blocks deposits: a 1 XRP deposit then returns tecLIMIT_EXCEEDED.

132BAF7C1B8208E695A453C0EE061BC5E3528FC97571589D8A2EA66F027B5B4F

Proposed fix. Add a VaultSet flag that closes a vault to deposits. Repeat “0 means no cap” on the VaultSet reference.

10 · Vault shares carry lsfMPTCanTrade

vault_info returns the share issuance with Flags: 56, which is lsfMPTCanEscrow (0x08), lsfMPTCanTrade (0x10) and lsfMPTCanTransfer (0x20). VaultCreate has no field that sets these flags. The xrpl.org MPTokenIssuance reference describes lsfMPTCanTrade as allowing holders to “trade their balances using the XRP Ledger DEX or AMM”. On the devnet, an OfferCreate with a share amount returned temDISABLED. The feature list contains MPTokensV1 and DynamicMPT, and no amendment for MPT trading.

Proposed fix. State on the MPTokenIssuance reference that DEX trading of MPTs needs an amendment not yet available, or leave lsfMPTCanTrade unset on vault shares until then.

Minimum bar, guardrails, escrow swap

The minimum bar run, the six guardrails, the closed-ended vault control and the escrow swap are listed with explorer links in the README. All 15 XLS-65/66 transaction types were submitted at least once, including a full tfLoanImpair then tfLoanDefault cycle.

What worked, limits, material

tecHAS_OBLIGATIONS names its cause on the three delete transactions. VaultDelete removes third-party MPToken objects and refunds their reserve. The cover floor is enforced to the drop, and cover rates return temINVALID when changed on an existing broker. The closed-ended vault phases behave as XLS-65 describes. Limits: XRP only, so neither clawback was tested with an issued asset; payment schedules of 60 to 90 seconds; one build. In the repo: scripts/raw-submit.mjs (counterparty signer), scripts/_probe-audit-hashes.mjs (hash audit), deck/capture-explorer.mjs (these screenshots). In passing: the Lending Protocol concept page spells PrincipleOutstanding and depositor; validateVaultCreate in xrpl@4.6.0 accepts any number for WithdrawalPolicy, and 0, 2, 3, 99 and 255 return temMALFORMED.